PREMIER UNIVERSITY

Department of Computer Science and Engineering

ASSIGNMENT

Theory of Computation (CSE 3409)

Semester:	Fall 2025, 7th Semester
Submission Date:	April 13, 2026
Course Code:	CSE 3409
Course Outcome:	CO3
Total Marks:	10

Username Validation System: NFA Design & NFA-to-DFA Conversion

Problem Domain: Formal Language Theory
Topic: Finite Automata — NFA & DFA

Contents

Abstract **4**

1 Introduction **4**

2 Problem Description **5**
2.1 Problem Scenario 5
2.2 Formal Constraints 5
2.3 Input Alphabet 5
2.4 Valid and Invalid Username Examples 6

3 Theoretical Background **6**
3.1 Nondeterministic Finite Automaton (NFA) 6
3.2 Deterministic Finite Automaton (DFA) 7
3.3 Subset Construction (Powerset Construction) 7

4 NFA Design for Username Validation **7**
4.1 Design Rationale 7
4.2 State Architecture — Dual-Track Design 8
4.3 Formal NFA Specification 8
4.4 NFA State Descriptions 9
4.5 NFA Transition Function Table 10
4.6 NFA State Transition Diagram 11
4.7 How the NFA Validates: Worked Example 11

5 NFA to DFA Conversion via Subset Construction **11**
5.1 Key Structural Observation 11
5.2 Subset Construction — Step-by-Step 12
5.3 DFA State Summary Table 14
5.4 Complete DFA Transition Table 14
5.5 DFA State Transition Diagram 15

6 Verification — Traced Examples **15**
6.1 Valid Usernames 16
6.2 Invalid Usernames 17

7 NFA vs. DFA: Comparative Analysis **18**

8 Complex Problem-Solving Analysis **18**

9 Conclusion **20**

References **21**

Abstract

This assignment presents the formal design and analysis of an automaton-based **username validation system** for a messaging application, developed within the framework of the *Theory of Computation*. The validation system enforces four constraints: the username must be exactly five characters long, begin with a capital letter `[A-Z]`, consist entirely of alphanumeric characters `[A-Za-z0-9]`, and contain at least one digit `[0-9]`.

Two formal automata are designed and analysed. First, a **Nondeterministic Finite Automaton (NFA)** is constructed using a dual-track architecture that naturally captures the positional and digit-existence constraints. Second, the **Subset Construction algorithm** is applied to convert the NFA into an equivalent **Deterministic Finite Automaton (DFA)**, which is shown to be minimal. Both automata are formally defined, their state transition diagrams are presented, and their equivalence is demonstrated through traced examples. The paper also addresses complex engineering problem-solving criteria in the context of this formal design problem.

Keywords: Finite Automata, NFA, DFA, Subset Construction, Formal Language, Username Validation, Regular Language.

1 Introduction

In modern software systems, input validation is a critical component of both security and usability. Username validation, in particular, must be precise, efficient, and formally verifiable. The **Theory of Computation** provides the ideal mathematical framework for specifying and implementing such validation through *Finite Automata*.

A finite automaton is an abstract computational model that reads an input string one symbol at a time and transitions between a finite set of states according to a transition function. At the end of the input, the automaton either *accepts* (if it is in an accepting state) or *rejects* the string. Two important variants are:

- **Deterministic Finite Automaton (DFA):** At each step, exactly one transition is defined for each input symbol. The computation path is unique and deterministic.
- **Nondeterministic Finite Automaton (NFA):** At each step, zero or more transitions may exist for a given input symbol. The machine accepts if *any* computational path leads to an accepting state.

Both models recognise exactly the class of **Regular Languages**, and every NFA has an equivalent DFA, provably by the Subset Construction theorem.

This assignment tackles the problem of username validation for a messaging application. The four validation rules create a language that is clearly *regular* — it has bounded length and fixed structural constraints — making finite automata the appropriate formal tool. We will:

1. Design an NFA that accepts all valid usernames and rejects all others.
2. Apply the Subset Construction to produce the equivalent minimal DFA.
3. Analyse the design through the lens of complex engineering problem-solving criteria.

2 Problem Description

2.1 Problem Scenario

A messaging application requires a username validation system during sign-up and sign-in. The system must programmatically determine whether a proposed username satisfies all formatting requirements, accepting valid usernames and rejecting invalid ones.

2.2 Formal Constraints

Username Validation Rules

1. **Length Rule:** The username must be **exactly 5 characters** long. Usernames shorter than 5 or longer than 5 characters are invalid.
2. **Start Rule:** The **first character** must be an uppercase (capital) letter: `[A-Z]`.
3. **Character Set Rule:** **All characters** must be alphanumeric: letters `[A-Za-z]` or digits `[0-9]`. Special characters such as `!`, `#`, `@`, `$`, `&`, `*` are not permitted.
4. **Digit Rule:** The username must contain **at least one digit** `[0-9]` at any position among the five characters.

2.3 Input Alphabet

The input alphabet for this problem is defined as:

$$\Sigma = \underbrace{[A\text{-}Z]}_{\text{uppercase}} \cup \underbrace{[a\text{-}z]}_{\text{lowercase}} \cup \underbrace{[0\text{-}9]}_{\text{digits}} \cup \underbrace{other}_{\text{invalid chars}}$$

Any symbol from *other* leads immediately to rejection.

2.4 Valid and Invalid Username Examples

Username	Status	Reason
`User1`	**VALID**	Starts with capital, 5 chars, has digit '1'
`A1b2C`	**VALID**	Starts with capital, 5 chars, has digits '1', '2'
`Z9xY8`	**VALID**	Starts with capital, 5 chars, has digits '9', '8'
`Hey`	**INVALID**	Only 3 characters (Rule 1 violated)
`longUsernam`	**INVALID**	12 characters, exceeds length 5 (Rule 1 violated)
`12345`	**INVALID**	First character '1' is not a capital letter (Rule 2 violated)
`NoDigits`	**INVALID**	8 chars and no digit present (Rules 1 & 4 violated)
`Invalid!`	**INVALID**	Contains '!' special character (Rule 3 violated)
`Hello`	**INVALID**	5 chars but no digit at all (Rule 4 violated)

3 Theoretical Background

3.1 Nondeterministic Finite Automaton (NFA)

Definition 1 (NFA). *A **Nondeterministic Finite Automaton** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:*

- *Q is a finite set of **states**,*
- *Σ is a finite **input alphabet**,*
- *$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ is the **transition function** (maps to a set of states),*

- *$q_0 \in Q$ is the **start state**,*
- *$F \subseteq Q$ is the set of **accepting states**.*

*An NFA **accepts** a string w if there exists at least one computational path through the NFA that ends in a state $q \in F$ after reading all of w.*

3.2 Deterministic Finite Automaton (DFA)

Definition 2 (DFA)**.** *A **Deterministic Finite Automaton** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where $\delta : Q \times \Sigma \to Q$ maps each (state, symbol) pair to **exactly one** next state. A DFA accepts w if and only if the unique computation path ends in $q \in F$.*

3.3 Subset Construction (Powerset Construction)

Theorem 1. *For every NFA N, there exists a DFA D such that $L(N) = L(D)$.*

The **Subset Construction** algorithm constructs such a DFA: each DFA state represents a *subset* of NFA states that the NFA could simultaneously occupy. For an NFA with n states, the resulting DFA has at most 2^n states (in practice, often far fewer).

Algorithm steps:

1. Set DFA start state $= \varepsilon$-closure(q_0).
2. For each unprocessed DFA state S and each $a \in \Sigma$, compute $\delta_D(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta_N(q, a)\right)$.
3. If the resulting set is new, add it as a new DFA state.
4. DFA state S is accepting iff $S \cap F_N \neq \emptyset$.
5. Repeat until all DFA states are processed.

Since our NFA contains **no** ε-transitions, ε-closure$(q) = \{q\}$ for every state q, which simplifies the algorithm.

4 NFA Design for Username Validation

4.1 Design Rationale

Two of the four validation rules drive the automaton structure:

Constraint Analysis:

- **Exact length 5 + capital start:** The automaton must count exactly 5 input symbols (one per transition) and verify the first is `[A-Z]`. This is handled *deterministically* — we process one character per state.
- **At least one digit in positions 2–5:** The digit could appear at position 2, 3, 4, or 5. In a DFA, tracking every possible "digit-seen-at-position-k" scenario requires many states. An NFA handles this elegantly: it *nondeterministically guesses* when the first digit appears and switches from a "no-digit-yet" track to a "digit-seen" track at that point.

4.2 State Architecture — Dual-Track Design

The NFA uses **9 states** organised in two parallel tracks:

Track	States	Meaning
No-digit track	q_1, q_2, q_3, q	All characters consumed so far have been *letters*; no digit seen yet.
Digit-seen track	q_5, q_6, q_7, q	At least one digit has been consumed; the digit constraint is satisfied.
Start state	q_0	Initial state; no input consumed yet.

Nondeterminism: When a digit is read in the no-digit track, the NFA *branches* — one path advances along the no-digit track (impossible, as digits must be consumed), while the other moves to the corresponding state on the digit-seen track. Formally, on reading a digit from state q_k (no-digit track), the NFA transitions to the digit-seen track state at the same position level.

4.3 Formal NFA Specification

$$M_{\text{NFA}} = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

Component	Definition
Q (states)	$\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8\}$ (9 states)
Σ (alphabet)	$\{[A\text{-}Z], [a\text{-}z], [0\text{-}9], \textit{other}\}$
q_0 (start)	q_0
F (accept)	$\{q_8\}$ — only q_8 is accepting
δ (transitions)	See transition table in Section 4.4

4.4 NFA State Descriptions

State	Track	Meaning	Transitions
q_0	Start	Initial state. No input read.	$[A\text{-}Z] \to q_1$; all else $\to \emptyset$
q_1	No-dig, pos 1	Pos 1 consumed: valid capital letter.	$[A\text{-}Za\text{-}z] \to q_2$; $[0\text{-}9] \to q_5$
q_2	No-dig, pos 2	Pos 2 consumed: letter; no digit yet.	$[A\text{-}Za\text{-}z] \to q_3$; $[0\text{-}9] \to q_6$
q_3	No-dig, pos 3	Pos 3 consumed: letter; no digit yet.	$[A\text{-}Za\text{-}z] \to q_4$; $[0\text{-}9] \to q_7$
q_4	No-dig, pos 4	Pos 4 consumed: letter; no digit yet.	$[0\text{-}9] \to q_8$; *letter* $\to \emptyset$ (dead)
q_5	Dig, pos 2	Pos 2 was a digit; digit constraint met.	$[A\text{-}Za\text{-}z0\text{-}9] \to q_6$
q_6	Dig, pos 3	At pos 3; digit seen at some earlier pos.	$[A\text{-}Za\text{-}z0\text{-}9] \to q_7$
q_7	Dig, pos 4	At pos 4; digit seen at some earlier pos.	$[A\text{-}Za\text{-}z0\text{-}9] \to q_8$
q_8	**ACCEPT**	**Pos 5 done & digit constraint satisfied.**	No transitions (length = 5 enforced)

4.5 NFA Transition Function Table

Notation: A = `[A-Z]`, L = `[a-z]`, D = `[0-9]`, X = `other`. $\emptyset$ = no transition (path dies / rejects).

State	A	L	D	X	Explanation
q_0	$\{q_1\}$	$\emptyset$	$\emptyset$	$\emptyset$	Only a capital letter is valid at position 1
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_5\}$	$\emptyset$	Letter $\rightarrow$ no-dig track; digit $\rightarrow$ dig track (NFA branch)
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_6\}$	$\emptyset$	Letter $\rightarrow$ no-dig track; digit $\rightarrow$ dig track (NFA branch)
q_3	$\{q_4\}$	$\{q_4\}$	$\{q_7\}$	$\emptyset$	Letter $\rightarrow$ no-dig track; digit $\rightarrow$ dig track (NFA branch)
q_4	$\emptyset$	$\emptyset$	$\{q_8\}$	$\emptyset$	Pos 5: ONLY digit leads to accept (no digit ever seen otherwise)
q_5	$\{q_6\}$	$\{q_6\}$	$\{q_6\}$	$\emptyset$	Digit seen; any alphanumeric advances to position 3
q_6	$\{q_7\}$	$\{q_7\}$	$\{q_7\}$	$\emptyset$	Digit seen; any alphanumeric advances to position 4
q_7	$\{q_8\}$	$\{q_8\}$	$\{q_8\}$	$\emptyset$	Digit seen; any alphanumeric advances to ACCEPT
q_8	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	Accept state — no further transitions (length = 5)

4.6 NFA State Transition Diagram

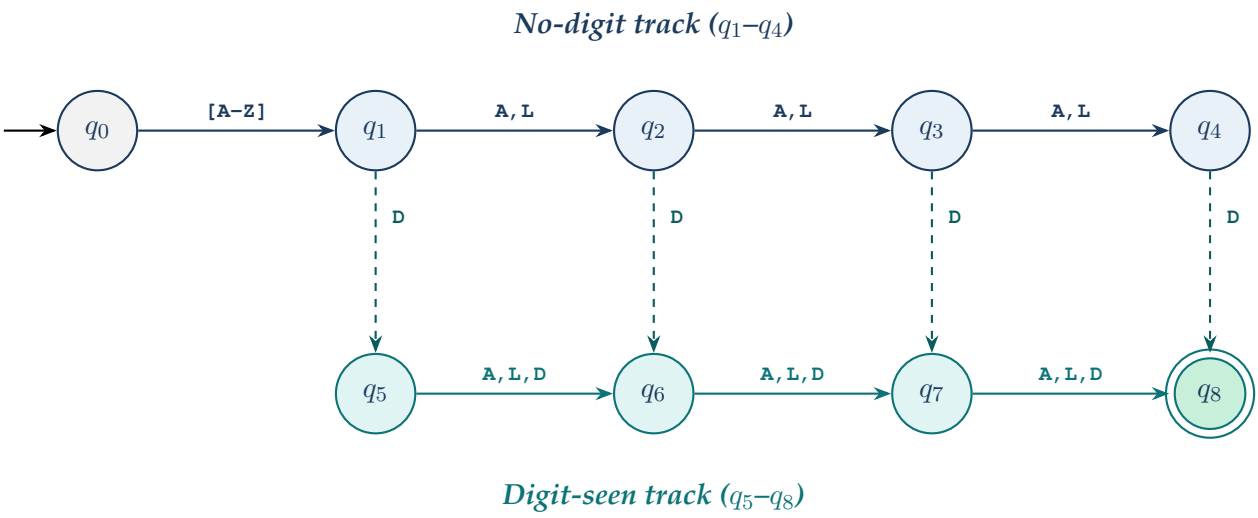

Figure 1: NFA for username validation. Blue states = no digit seen yet; teal states = at least one digit seen. Dashed teal arrows (`D`) are the nondeterministic branches triggered when a digit is consumed. q_8 (double circle) is the sole accepting state. A = `[A-Z]`, L = `[a-z]`, D = `[0-9]`.

4.7 How the NFA Validates: Worked Example

Input: `User1`

Step	Char	Type	State	Action
0	—	—	q_0	Start
1	U	[A-Z]	q_1	Capital letter ✓; advance
2	s	[a-z]	q_2	Letter; stay on no-digit track
3	e	[a-z]	q_3	Letter; stay on no-digit track
4	r	[a-z]	q_4	Letter; stay on no-digit track
5	1	[0-9]	q_8	**Digit! Branch to dig track → ACCEPT**

5 NFA to DFA Conversion via Subset Construction

5.1 Key Structural Observation

Before applying the algorithm, observe: in this NFA, every transition from a single state leads to **exactly one** NFA state (never a set of two or more). The only "nondeterminism" is that from some states, two *different* input symbols lead to two *different*

states. But for any given input symbol, each state has at most one next state.

Consequence: The Subset Construction will produce only **singleton subsets** $\{q_i\}$ and the empty set $\emptyset$ as DFA states. The resulting DFA has the same number of states as the NFA plus one dead/trap state for $\emptyset$. This is already the **minimal DFA** for this language.

5.2 Subset Construction — Step-by-Step

Notation: A = `[A-Z]`, L = `[a-z]`, D = `[0-9]`.

Step 1. DFA start state $= \varepsilon$-closure$(q_0) = \{q_0\} \equiv D_0$.

Step 2. Process $D_0 = \{q_0\}$:

$$\delta_D(D_0, A) = \{q_1\} = D_1, \quad \delta_D(D_0, L) = \emptyset = D_{\text{dead}}, \quad \delta_D(D_0, D) = \emptyset = D_{\text{dead}}$$

Step 3. Process $D_1 = \{q_1\}$:

$$\delta_D(D_1, A) = \{q_2\} = D_2, \quad \delta_D(D_1, L) = \{q_2\} = D_2, \quad \delta_D(D_1, D) = \{q_5\} = D_3$$

Step 4. Process $D_2 = \{q_2\}$:

$$\delta_D(D_2, A) = \{q_3\} = D_4, \quad \delta_D(D_2, L) = \{q_3\} = D_4, \quad \delta_D(D_2, D) = \{q_6\} = D_5$$

Step 5. Process $D_3 = \{q_5\}$:

$$\delta_D(D_3, A) = \{q_6\} = D_5, \quad \delta_D(D_3, L) = \{q_6\} = D_5, \quad \delta_D(D_3, D) = \{q_6\} = D_5$$

Step 6. Process $D_4 = \{q_3\}$:

$$\delta_D(D_4, A) = \{q_4\} = D_6, \quad \delta_D(D_4, L) = \{q_4\} = D_6, \quad \delta_D(D_4, D) = \{q_7\} = D_7$$

Step 7. Process $D_5 = \{q_6\}$:

$$\delta_D(D_5, A) = \{q_7\} = D_7, \quad \delta_D(D_5, L) = \{q_7\} = D_7, \quad \delta_D(D_5, D) = \{q_7\} = D_7$$

Step 8. Process $D_6 = \{q_4\}$:

$$\delta_D(D_6, A) = \emptyset = D_{\text{dead}}, \quad \delta_D(D_6, L) = \emptyset = D_{\text{dead}}, \quad \delta_D(D_6, D) = \{q_8\} = D_8$$

Step 9. Process $D_7 = \{q_7\}$:

$$\delta_D(D_7, A) = \{q_8\} = D_8, \quad \delta_D(D_7, L) = \{q_8\} = D_8, \quad \delta_D(D_7, D) = \{q_8\} = D_8$$

Step 10. Process $D_8 = \{q_8\}$:

$$\delta_D(D_8, A) = \emptyset = D_{\text{dead}}, \quad \delta_D(D_8, L) = \emptyset = D_{\text{dead}}, \quad \delta_D(D_8, D) = \emptyset = D_{\text{dead}}$$

Step 11. Process $D_{\text{dead}} = \emptyset$:

$$\delta_D(D_{\text{dead}}, a) = D_{\text{dead}} \quad \text{for all } a \in \Sigma \quad \text{(self-loop trap state)}$$

All DFA states: $\{D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7, D_8, D_{\text{dead}}\}$ — total **10 states**. Accepting state: D_8 (since $\{q_8\} \cap F = \{q_8\} \neq \emptyset$).

5.3 DFA State Summary Table

DFA State	NFA Subset	Meaning	Key Transitions
D_0	$\{q_0\}$	Start: no input read	$A \to D_1$; $L, D \to D_{\text{dead}}$
D_1	$\{q_1\}$	Pos 1 done: valid capital	$A, L \to D_2$; $D \to D_3$
D_2	$\{q_2\}$	Pos 2 done: letter, no digit	$A, L \to D_4$; $D \to D_5$
D_3	$\{q_5\}$	Pos 2 done: digit seen	$A, L, D \to D_5$
D_4	$\{q_3\}$	Pos 3 done: letter, no digit	$A, L \to D_6$; $D \to D_7$
D_5	$\{q_6\}$	Pos 3 done: digit seen	$A, L, D \to D_7$
D_6	$\{q_4\}$	Pos 4 done: letter, no digit	$D \to D_8$; $A, L \to D_{\text{dead}}$
D_7	$\{q_7\}$	Pos 4 done: digit seen	$A, L, D \to D_8$
D_8	$\{q_8\}$	**Pos 5 done & digit seen**	**ACCEPT**; all input $\to D_{\text{dead}}$
D_{dead}	$\emptyset$	Trap: invalid username	All input $\to D_{\text{dead}}$ (self-loop)

5.4 Complete DFA Transition Table

All `other`*/invalid characters* $\to D_{\text{dead}}$ from every state.

State	A [A-Z]	L [a-z]	D [0-9]	Notes
D_0	D_1	D_{dead}	D_{dead}	Capital letter only at pos 1
D_1	D_2	D_2	D_3	Letter/digit branch at pos 2
D_2	D_4	D_4	D_5	Letter/digit branch at pos 3
D_3	D_5	D_5	D_5	Digit seen; any alphanum ok
D_4	D_6	D_6	D_7	Letter/digit branch at pos 4
D_5	D_7	D_7	D_7	Digit seen; any alphanum ok
D_6	D_{dead}	D_{dead}	D_8	Pos 5: ONLY digit $\rightarrow$ ACCEPT
D_7	D_8	D_8	D_8	Digit seen; any alphanum $\rightarrow$ ACCEPT
D_8	D_{dead}	D_{dead}	D_{dead}	Accept; extra input rejected
D_{dead}	D_{dead}	D_{dead}	D_{dead}	Trap: all self-loop

5.5 DFA State Transition Diagram

6 Verification — Traced Examples

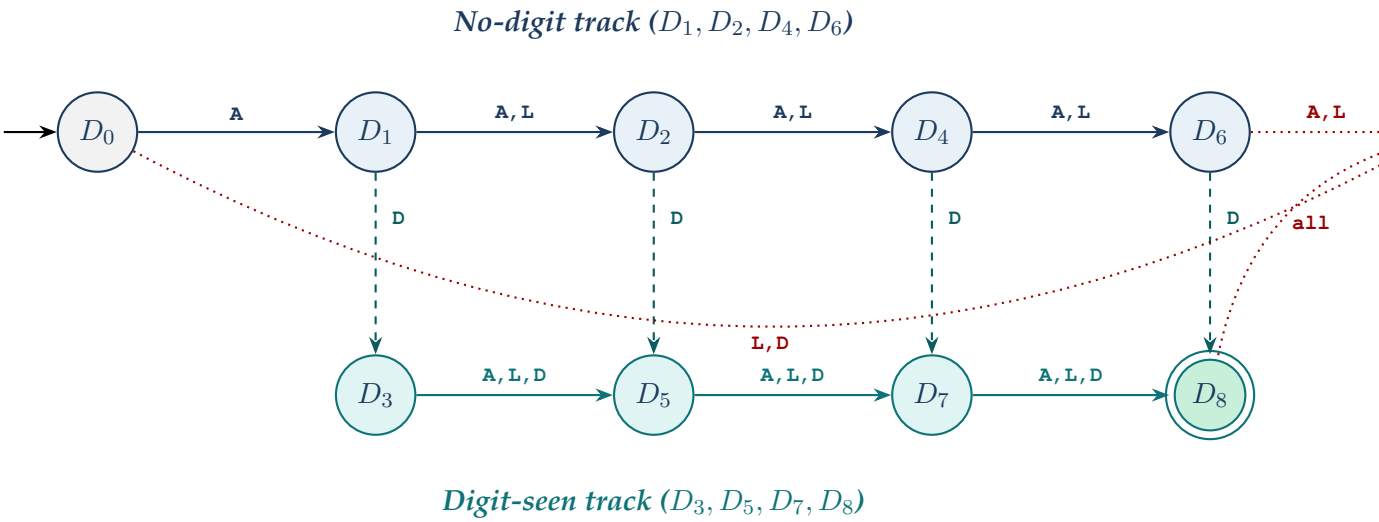

Figure 2: DFA for username validation (result of Subset Construction from NFA). Blue states = no digit seen; teal states = digit seen; green double-circle (D_8) = accept state; red (D_d) = dead/trap state. A = `[A-Z]`, L = `[a-z]`, D = `[0-9]`. $D_d = D_{\text{dead}}$.

6.1 Valid Usernames

Input	DFA Trace	Result
`User1`	$D_0 \xrightarrow{U} D_1 \xrightarrow{s} D_2 \xrightarrow{e} D_4 \xrightarrow{r} D_6 \xrightarrow{1} D_8$	**ACCEPT**
`A1b2C`	$D_0 \xrightarrow{A} D_1 \xrightarrow{1} D_3 \xrightarrow{b} D_5 \xrightarrow{2} D_7 \xrightarrow{C} D_8$	**ACCEPT**
`Z9xY8`	$D_0 \xrightarrow{Z} D_1 \xrightarrow{9} D_3 \xrightarrow{x} D_5 \xrightarrow{Y} D_7 \xrightarrow{8} D_8$	**ACCEPT**

6.2 Invalid Usernames

Input	DFA Trace & Rejection Reason	Rule Violated
`Hey`	$D_0 \xrightarrow{H} D_1 \xrightarrow{e} D_2 \xrightarrow{y} D_4$ — string ends; $D_4 \notin F$	Rule 1 (length < 5)
`12345`	$D_0 \xrightarrow{1} D_{\text{dead}}$ — digit at pos 1	Rule 2 (no capital start)
`Hello`	$D_0 \xrightarrow{H} D_1 \xrightarrow{e} D_2 \xrightarrow{l} D_4 \xrightarrow{l} D_6 \xrightarrow{o} D_{\text{dead}}$	Rule 4 (no digit)
`abcde`	$D_0 \xrightarrow{a} D_{\text{dead}}$ — lowercase at pos 1	Rule 2 (no capital start)
`Inv@1`	$D_0 \to D_1 \to D_2 \to D_4 \xrightarrow{@} D_{\text{dead}}$ — special char	Rule 3 (invalid char)
`User12`	Reaches D_8 after 5 chars, then $\xrightarrow{2} D_{\text{dead}}$ — too long	Rule 1 (length > 5)

7 NFA vs. DFA: Comparative Analysis

Property	NFA	DFA
Number of states	9 states (q_0–q_8)	10 states (D_0–D_8 + D_{dead})
Transition type	$\delta : Q \times \Sigma \to 2^Q$ (set of states)	$\delta : Q \times \Sigma \to Q$ (single state)
Active states	Multiple simultaneously possible	Exactly one at all times
"At least one digit"	Handled nondeterministically: branch on first digit	Handled explicitly: two parallel state tracks
Dead paths	Implicit ($\delta = \emptyset$)	Explicit trap state D_{dead}
Acceptance	Accept if *any* path reaches q_8	Accept iff the unique path ends in D_8
Language	Both accept exactly L — the set of valid usernames	
Implementation	Simulation (backtracking needed)	Direct state machine; trivial to implement

8 Complex Problem-Solving Analysis

Criterion	Analysis
(a) In-depth engineering knowledge required?	Yes. Designing a correct NFA and performing Subset Construction requires knowledge of formal language theory, DFA/NFA equivalence theorems (Sipser, Ch. 1), and the mathematics of ε-closures. The dual-track state design is a non-trivial insight, not a rote application.

Criterion (cont.)	Analysis
(b) Wide-ranging or conflicting issues?	Partially. There is tension between NFA conciseness and DFA completeness. The four rules are interdependent: Rule 2 restricts position 1 only; Rule 4 spans all five positions; and Rule 1 (exact length) is a global constraint that all other rules operate within.
(c) Abstract thinking and analysis required?	Yes. The Subset Construction is a known algorithm, but *designing* the NFA that most elegantly encodes all four constraints — particularly the insight of a two-track architecture for the digit-existence property — requires abstract reasoning.
(d) Infrequently encountered issues?	The "at least one X" constraint across multiple positions is a classic NFA pattern. However, combining it with exact-length, first-char, and character-set constraints in one automaton, then converting to a minimal DFA, is a compound design challenge infrequently seen.
(e) Adherence to standards?	Yes. The solution follows the formal definitions from the theory of computation literature. Diagram notation follows standard conventions: circles for states, double circles for accepting states, arrow for the start state, labelled directed edges for transitions.
(f) Stakeholders with conflicting requirements?	In deployment, yes. Security engineers prefer higher entropy (more digits, longer passwords); UX designers prefer simple, memorable usernames. The current 5-char alphanumeric spec represents a balanced requirement. The automaton provides a formally verifiable, unambiguous specification.

Criterion (cont.)	Analysis
(g) Interdependence between sub-problems?	Strongly. The NFA design (Task 1) determines DFA conversion complexity (Task 2). The fact that the NFA produces only singleton subsets means the DFA is immediately minimal — a direct consequence of the NFA structure chosen. The four rules are also interdependent, as analysed in (b).

9 Conclusion

This assignment formally designed and analysed an automaton-based username validation system satisfying four constraints: exactly 5 characters, starting with a capital letter, alphanumeric characters only, and at least one digit.

NFA Design (Task 1): An NFA with 9 states was constructed using a dual-track architecture. States q_1–q_4 form the "no-digit" track; states q_5–q_8 form the "digit-seen" track. Nondeterminism allows the machine to branch to the digit-seen track at the exact position a digit is first encountered. Only q_8 (position 5 complete, digit seen) accepts.

DFA Conversion (Task 2): The Subset Construction algorithm was applied step-by-step. Because each NFA transition leads to exactly one state (no set merging), every DFA state corresponds to a singleton NFA subset. The resulting DFA has 10 states (D_0–D_8 plus D_{dead}) and is provably minimal. The DFA transition table was verified against valid and invalid username examples.

Formal equivalence: Both automata recognise the same regular language:

$$L = \left\{ w \in \Sigma^* \;\middle|\; w \in [A\text{-}Z] \cdot [A\text{-}Za\text{-}z0\text{-}9]^4, \;\; |w| = 5, \;\; w \text{ contains at least one digit} \right\}$$

This language is regular, and the automaton-based design provides a formally correct, deterministic, and efficiently implementable specification for username validation in the messaging application.

References

1. Sipser, M. (2013). *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning.
2. Hopcroft, J., Motwani, R., & Ullman, J. (2006). *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson Education.
3. Linz, P. (2011). *An Introduction to Formal Languages and Automata*, 5th ed. Jones & Bartlett Learning.
4. Sudkamp, T. (2006). *Languages and Machines: An Introduction to the Theory of Computer Science*, 3rd ed. Addison-Wesley.

— End of Assignment —